

SHILPA3D

Preserving Art in Three Dimensions

3D Reconstruction of Statues Using NeRF and Gaussian Splatting

Project Documentation — Updated Edition

Backend data layer: PostgreSQL + Sequelize (supersedes earlier MongoDB plan)

Duration	3 Months
Team Size	3 Members
Domain	AI / Computer Vision / Full-Stack Web Development
Institution	Gandaki College of Engineering and Science, Pokhara University
Reconstruction Methods	COLMAP (SfM) → NeRF (nerfstudio) and 3D Gaussian Splatting
Data Layer	PostgreSQL 15+ with Sequelize ORM (migrated from the original MongoDB plan)
AI-to-Backend Bridge	FastAPI (Python, on Colab) exposed to the Node.js backend via an ngrok tunnel

1. Introduction

Shilpa3D is a research-oriented 3D reconstruction system that generates realistic 3D digital models of statues from ordinary 2D photographs. The project applies two of the most significant neural rendering techniques in modern computer vision — Neural Radiance Fields (NeRF) and 3D Gaussian Splatting (3DGS) — and packages them behind a full-stack web application so a non-technical user can upload photos and receive an interactive 3D result without touching a command line.

Rather than depending on expensive specialized hardware such as LiDAR scanners or structured-light sensors, Shilpa3D operates entirely on multi-view photographs captured with an ordinary camera or phone. This makes it an accessible, cost-effective, and scalable solution — particularly valuable for cultural heritage preservation, museum digitization, and academic research, and specifically for digitizing Nepali cultural statues, an area with no prior published NeRF/3DGS application.

1.1 What Shilpa3D Does

- Accepts a set of multi-view photographs of a statue as input, uploaded through an authenticated web session.
- Estimates camera positions and orientations for every photograph automatically using Structure-from-Motion (COLMAP).
- Applies two independent neural reconstruction pipelines — NeRF and 3D Gaussian Splatting — to the same camera-posed dataset.
- Exports a point cloud (.ply) and rendered novel views from each method, and scores both against held-out ground-truth photos using PSNR, SSIM, and LPIPS.
- Persists every user, upload, reconstruction session, and result in a PostgreSQL database so history and results can be revisited later.
- Displays and compares results through an interactive React + Three.js web interface.

1.2 Interdisciplinary Nature

Field	Contribution
Computer Vision	Extracting 3D structure (camera poses, sparse geometry) from 2D photographs via COLMAP.
Deep Learning	Neural networks (MLPs, differentiable Gaussian rasterization) that model volumetric and explicit scenes.
Neural Rendering	AI-based photorealistic image synthesis from learned scene representations.
3D Graphics	Point clouds, Gaussian primitives, and real-time rasterized rendering.
Backend Engineering	Session orchestration, authentication, and relational data modelling (Node.js, Express, PostgreSQL).
Web Development	Full-stack interface for upload, progress tracking, and interactive 3D visualization.

2. Abstract

The digital reconstruction of real-world objects into accurate 3D models is a fundamental problem in computer vision, with applications spanning cultural heritage preservation, virtual reality, and digital archiving. Traditional approaches typically require costly specialized equipment or dense sensor data, limiting accessibility for smaller institutions and independent researchers — a constraint that is especially consequential for cultural heritage work in resource-limited settings such as Nepal.

Shilpa3D addresses this challenge by generating 3D models of statues using only a set of 2D photographs captured from different viewpoints. Camera geometry is automatically recovered using Structure-from-Motion (COLMAP) before reconstruction begins. The recovered poses and sparse point cloud then feed two independent reconstruction pipelines: Neural Radiance Fields (NeRF, via nerfstudio's nerfacto model) and 3D Gaussian Splatting (the original Inria implementation).

Both methods are evaluated and compared across training time, GPU memory consumption, and quantitative image-quality metrics — PSNR, SSIM, and LPIPS — computed against images deliberately held out of training. Across both statues tested, Gaussian Splatting outperformed NeRF on every metric while training in roughly a third of the time (~35 minutes vs. ~90 minutes on a T4 GPU), making it the recommended default reconstruction method for this dataset scale.

The complete system is exposed through a web-based interface built with React.js (Vite) and Three.js on the frontend, Node.js + Express.js as the application backend, PostgreSQL (via Sequelize) as the persistence layer, and Passport.js + JWT for authentication (local and Google OAuth2). The Python reconstruction pipeline runs on a Google Colab GPU runtime behind a FastAPI server, tunnelled to the Node.js backend through ngrok — allowing free-tier GPU compute to be safely bridged into a production-style web application without the backend itself needing GPU access.

3. PROBLEM STATEMENT & OBJECTIVES

3. Problem Statement

The acquisition of accurate 3D models of physical objects has traditionally depended on specialized and expensive scanning hardware:

- LiDAR scanners — high cost, bulky, requires technical expertise.
- Structured-light scanners — sensitive to ambient light, limited portability.
- Depth cameras — lower resolution, not suitable for fine surface details.

Beyond hardware, several algorithmic challenges make image-based reconstruction difficult:

Challenge	Description
Camera Pose Estimation	Determining where each photo was taken relative to the subject, with high precision.
Rendering Quality	Producing photorealistic novel views, not just coarse geometry.
Scene Ambiguity	Resolving occluded regions and reflective or textureless surfaces — a known failure mode for small, texture-poor sculptural surfaces.
Computational Load	Neural rendering is GPU-intensive and time-consuming, especially on free-tier cloud GPUs.
Scalability	Systems must work within limited academic hardware budgets, and tolerate session interruptions.
Session Persistence	Every upload, reconstruction run, and result must be durably tracked per user, not just left as files on disk.

4. Objectives

4.1 Main Objective

To develop an intelligent, accessible 3D reconstruction system that generates realistic 3D models of statues from multi-view 2D photographs — without any specialized scanning hardware — using NeRF and Gaussian Splatting, backed by a persistent, multi-user web platform.

4.2 Specific Objectives

- Capture and prepare multi-view image datasets of statues under controlled conditions.
- Perform automatic camera pose estimation using Structure-from-Motion (COLMAP).
- Implement a NeRF-based 3D reconstruction pipeline using nerfstudio / PyTorch.
- Implement a 3D Gaussian Splatting reconstruction pipeline in PyTorch.
- Evaluate and compare both methods using quantitative (PSNR/SSIM/LPIPS) and qualitative visual analysis.
- Expose the reconstruction pipeline through a FastAPI service, bridged to a Node.js/Express.js REST API via ngrok.
- Persist users, uploads, sessions, and results in a relational PostgreSQL database via Sequelize.
- Implement authentication (local + Google OAuth2) with JWT-based session management.

- Build a React.js + Three.js frontend for interactive 3D model visualization and session history.

5. TEAM STRUCTURE & RESPONSIBILITIES

5. Team Structure & Responsibilities

Shilpa3D is developed by a team of three members, each owning a distinct technical domain:

Member	Role	Responsibilities
Member 1	AI & Reconstruction Lead	Dataset preparation, COLMAP pose estimation, NeRF pipeline, Gaussian Splatting pipeline, pipeline.py hardening, evaluation metrics.
Member 2	Backend Developer	Node.js + Express.js REST API, PostgreSQL schema and Sequelize models, Passport.js authentication, FastAPI/ngrok bridge integration, session orchestration.
Member 3	Frontend Developer	React.js (Vite) UI, Three.js 3D viewer integration, multi-image upload, processing dashboard, results and history dashboard.

6. System Overview

Shilpa3D is architecturally divided into three major subsystems that communicate over HTTP:

6.1 Web Application Layer (Node.js + Express.js + PostgreSQL)

Handles everything user-facing except the actual reconstruction math: authentication, image upload, project/session bookkeeping, progress polling, and result serving. All persistent state — users, image sets, reconstruction sessions, and results — lives in PostgreSQL, accessed through Sequelize models. This layer never touches a GPU.

6.2 AI Reconstruction Pipeline (Python, FastAPI, Colab GPU)

The computationally intensive core. It runs on a Google Colab T4 GPU runtime as a FastAPI service (see Section 9) and performs image preprocessing, COLMAP pose estimation, NeRF and Gaussian Splatting training, export, and quantitative evaluation. Because Colab cannot be reached directly from the public internet, an ngrok tunnel exposes the FastAPI server's port so the Node.js backend can call it as if it were any other internal microservice.

6.3 Web-Based Visualization Platform (React.js + Three.js)

The frontend communicates with the Node.js backend over HTTP/JSON. It never talks to the FastAPI/Colab service directly — the Node backend is the single point of contact, which keeps the ngrok URL, retry logic, and long-running job orchestration out of the browser. Reconstructed point clouds are rendered interactively using Three.js.

6.4 End-to-End Workflow

Step	Stage	Description
1	Sign Up / Log In	User authenticates via local email+password or Google OAuth2 (Passport.js). Node.js issues a JWT session.
2	Image Collection	Photographs of the statue are taken from 20–40+ viewpoints, maintaining consistent lighting and 60–80% overlap.
3	Upload	Frontend uploads the image set to Node.js (Multer), which creates an ImageSet row in PostgreSQL and forwards a job request to the FastAPI pipeline.
4	Camera Pose Estimation	COLMAP's Structure-from-Motion pipeline detects feature matches between images and recovers camera poses plus a sparse point cloud.
5	NeRF Reconstruction	nerfstudio's nerfacto model learns a continuous volumetric representation of the scene, trains, exports a .ply point cloud, and renders novel views.
6	Gaussian Splatting	The sparse point cloud from COLMAP initializes millions of 3D Gaussian primitives, optimized against the training images and rasterized to produce renders.
7	Evaluation	PSNR, SSIM, and LPIPS are computed for both methods against held-out test images; training time and GPU memory are logged.

Step	Stage	Description
8	Persist Results	Node.js polls FastAPI's status/results endpoints and writes ModelOutput, RenderedView, and Metrics rows into PostgreSQL.
9	Frontend Visualization	Results are served via the Node.js backend and displayed interactively in the browser using Three.js, alongside the metrics comparison table.

7. CORE COMPONENTS

7. Core Components

The system is composed of nine integrated components. Each is described below in terms of its purpose, process, inputs, outputs, and technologies used.

Component 1 — Dataset Collection

Purpose: prepare a set of multi-view images used as input to the reconstruction pipeline. High-quality reconstruction requires photographs that capture the subject from enough angles to infer its 3D geometry; poor image quality (blur, harsh shadows, inconsistent exposure) directly degrades reconstruction output.

Capture Guidelines

- Capture 20–40+ images per statue (the tested pipeline used 15–21 images per smoke/full test; more images improves camera registration density).
- Walk in a circular arc around the subject at multiple elevation levels.
- Maintain consistent, diffuse lighting — avoid harsh shadows.
- Ensure significant overlap between consecutive images (60–80%).
- Avoid motion blur — use a tripod if possible.
- Keep the background uncluttered to reduce ambiguity for COLMAP.

Output: an organized folder of .jpg/.png images, grouped per statue subject and per user session.

Component 2 — Image Preprocessing

Purpose: clean and normalize the raw image dataset before it enters the reconstruction pipeline.

Operation	Description
EXIF Normalization	Phone photos carry EXIF orientation tags that COLMAP and nerfstudio's dataloader can interpret differently, causing a width/height mismatch that crashes NeRF's data-loading workers. Images are re-saved with orientation baked into the pixel data (PIL ImageOps.exif_transpose) before training.
Blur Detection & Removal	Blurry images are identified using Laplacian variance and discarded from the dataset.
Brightness Normalization	Histogram equalization reduces lighting inconsistencies across frames.
Dataset Organization	Images are renamed systematically and stored under a per-session directory structure that both COLMAP and the web backend can address consistently.

Technologies: Python, OpenCV, Pillow (PIL).

Component 3 — Camera Pose Estimation (COLMAP)

Purpose: recover the position and orientation of each camera at the moment its photo was taken. Both NeRF and Gaussian Splatting require this camera pose for every input image; it is not embedded in a photograph and must be computed via Structure-from-Motion (SfM).

How COLMAP Works in the Pipeline

- Feature Extraction — colmap feature_extractor detects SIFT keypoints in every image, forced to the SIMPLE_PINHOLE camera model (the Gaussian Splatting rasterizer failed on COLMAP's default camera models).
- Feature Matching — colmap exhaustive_matcher (or sequential_matcher above ~100 images) finds matching keypoints across overlapping images.
- Sparse Reconstruction — colmap mapper triangulates 3D points and estimates all camera poses through bundle adjustment, writing sparse/0.
- Format Conversion — colmap model_converter exports the binary model to TEXT format so a custom Python script can parse cameras.txt and images.txt into nerfstudio's transforms.json.

A known environment issue: COLMAP fails to launch in the headless Colab environment because its default Qt display backend requires a GUI. This is resolved by setting QT_QPA_PLATFORM=offscreen before every pipeline run.

Output: transforms.json (camera intrinsics + per-image extrinsics) and a sparse point cloud — both consumed by the NeRF and Gaussian Splatting stages.

Component 4 — NeRF Reconstruction

Purpose: train a neural network to represent the statue as a continuous 3D volumetric scene, enabling photorealistic rendering from arbitrary viewpoints.

What is NeRF?

Neural Radiance Field (NeRF) encodes an entire 3D scene inside the weights of a Multi-Layer Perceptron (MLP). Instead of storing explicit geometry, it learns an implicit function that, for any 3D point and viewing direction, predicts RGB color and volume density.

NeRF Pipeline (as implemented)

- Input: multi-view images + camera poses (transforms.json) from COLMAP, images EXIF-normalized and copied into nerf_data/images.
- Training: ns-train nerfacto --data <nerf_data> --output-dir <nerf_out> --max-num-iterations <N> --steps-per-save <S> --vis tensorboard. Iteration count is 30,000 for a full run, 300 for a quick smoke test. --vis tensorboard logs locally with no external account required (an earlier version used --vis viewer+wandb, which crashed on a missing Weights & Biases API key).
- Checkpointing: a background watcher thread polls the output directory every 5 seconds and logs each new .ckpt file the instant it is written, so training progress is never a silent black box.
- Rendering: ns-render dataset --split train --output-format images renders novel views as image frames only, skipping video/mp4 encoding — an earlier version hung indefinitely muxing an mp4 after frames had already finished rendering.
- Export: ns-export pointcloud --num-points 1000000 --remove-outliers True --normal-method open3d produces a .ply point cloud for the website, equivalent in role to Gaussian Splatting's .ply output.
- Evaluation: ns-eval --load-config <config.yml> --output-path nerf_metrics.json computes PSNR, SSIM, and LPIPS against the eval-split images nerfstudio automatically holds out during training.

Advantages

- Photorealistic renders with fine surface detail and smooth geometry.
- Captures view-dependent lighting effects (reflections, specular highlights).
- No explicit mesh required — the scene is entirely implicit.

Limitations Observed in Practice

- Long training time — roughly 90 minutes for 30,000 iterations on a free-tier T4 GPU.

- Sensitive to full-resolution eval renders, which can exhaust GPU memory (observed CUDA OutOfMemoryError during periodic eval on 3024×4032 source images; mitigated by reducing eval image resolution / eval batch size).
- On the tested statue dataset, NeRF converged to noticeably lower PSNR/SSIM than Gaussian Splatting at the same wall-clock budget (see Section 13).

Technologies: Python, PyTorch, nerfstudio, CUDA.

Component 5 — Gaussian Splatting Reconstruction

Purpose: represent and render the 3D scene using millions of small Gaussian primitives — a faster, explicit alternative to NeRF.

What is 3D Gaussian Splatting?

3D Gaussian Splatting (3DGS) models a scene as a collection of 3D Gaussian distributions — small ellipsoidal blobs scattered through space. Each Gaussian stores a position (mean), covariance (shape/orientation), color (spherical-harmonic coefficients), and opacity.

Gaussian Splatting Pipeline (as implemented)

- Environment setup: `ensure_gaussian_splatting()` auto-clones the official Inria repository with `--recursive` submodules if missing, then builds the two required CUDA extensions directly — `pip install submodules/diff-gaussian-rasterization submodules/simple-knn` — rather than `pip install -e .` against the repo root, which has no `setup.py` and fails.
- Initialization: the sparse point cloud from COLMAP initializes Gaussian positions. Critically, `-s` (source path) must point at the COLMAP root folder (the parent of `sparse/0`), not `sparse/0` itself — pointing one level too deep produces 'Could not recognize scene type!'.
- Training: `python train.py -s <colmap_root> --images <images> -m <output> --iterations <N> --eval`. The `--eval` flag holds out a test split so PSNR/SSIM/LPIPS can later be computed against unseen views. 15,000 iterations for a full run, 300 for a smoke test.
- Checkpoint watching: the same live-streaming + checkpoint-watcher pattern used for NeRF logs each `point_cloud.ply` the moment it's written.
- Rendering: `python render.py -m <output> --iteration <N>` renders both train and test splits.
- Export: the trained `point_cloud/iteration_<N>/point_cloud.ply` is copied to the results folder as `<statue>.ply` for the website.
- Evaluation: `python metrics.py -m <output>` computes PSNR, SSIM, and LPIPS from the train/test renders, writing `results.json` (requires the `lpips` pip package, installed alongside the CUDA submodules).

Advantages

- Significantly faster rendering — real-time frame rates achievable with GPU rasterization.
- Lower computational cost than ray-marching through a neural network.
- Converges faster than NeRF — roughly a third of the training time on the tested dataset.

Limitations

- Sensitive to the quality and completeness of the initial COLMAP point cloud.
- Floating-artifact streaks appear in under-constrained regions with fewer overlapping camera views (observed near statue bases in both test statues).
- Large numbers of Gaussians can consume significant GPU memory during training.

Technologies: Python, PyTorch, CUDA (custom `diff-gaussian-rasterization` and `simple-knn` extensions).

Component 6 — Evaluation & Comparison

Purpose: quantitatively and qualitatively compare NeRF and Gaussian Splatting outputs to understand the tradeoffs of each approach, using a fixed, explicitly reported evaluation protocol (dataset, train/test split, iteration budget, and hardware) — a requirement identified from the comparative-benchmarking literature.

Metric	Type	Description
PSNR	Image Quality	Peak Signal-to-Noise Ratio. Pixel-level fidelity between rendered output and ground truth. Higher is better (dB).
SSIM	Image Quality	Structural Similarity Index. Perceptual similarity in luminance, contrast, and structure. Range 0–1; higher is better.
LPIPS	Perceptual	Learned Perceptual Image Patch Similarity. Deep-network-based perceptual quality. Lower is better.
Training Time	Efficiency	Total wall-clock time to train to the configured iteration budget.
Rendering FPS	Efficiency	Frames per second achievable at inference — reflects real-time usability.
GPU Memory	Efficiency	Peak GPU VRAM consumed during training and rendering (GB).

Component 7 — Backend (Node.js + Express.js)

Purpose: serve as the bridge between the frontend, the persistent PostgreSQL data layer, and the Python/FastAPI reconstruction service. All heavy AI computation runs remotely in Colab; the Node backend's job is authentication, orchestration, persistence, and file serving.

Key Responsibilities

- Authenticate users (Passport.js: local strategy + Google OAuth2) and issue JWT access/refresh tokens.
- Receive multi-image uploads from the frontend via HTTP POST (Multer), and create the corresponding ImageSet row.
- Forward the reconstruction request to the FastAPI pipeline over the ngrok tunnel, creating a ReconSession row.
- Poll the FastAPI status/results endpoints and update the ReconSession, ModelOutput, RenderedView, and Metrics rows in PostgreSQL as the pipeline progresses.
- Enforce session ownership — a user can only view, poll, or delete their own sessions (403 otherwise).
- Serve the generated 3D model files and rendered images back to the frontend, with file-path traversal protection.

Core API Endpoints (Node.js — application layer)

Endpoint	Description
POST /api/auth/register	Create a local account (email, username, password). Password hashed with bcryptjs.
POST /api/auth/login	Authenticate via Passport local strategy; returns a JWT access + refresh token pair.
GET /api/auth/google	Begin Google OAuth2 login flow (Passport google strategy).
GET /api/auth/google/callback	OAuth2 callback; creates/links the user record and issues JWT tokens.

Endpoint	Description
POST /api/uploads	Accepts a multi-image upload (multipart/form-data via Multer). Creates an ImageSet row and stores files.
POST /api/reconstruct	Starts a reconstruction job for an ImageSet. Creates a ReconSession row and forwards the job to FastAPI.
GET /api/sessions/:id/status	Returns the current pipeline stage and percentage (proxied/cached from FastAPI's status endpoint).
GET /api/sessions/:id/results	Returns the persisted ModelOutput + Metrics rows for a completed session.
GET /api/sessions/:id/files/:filename	Streams a result file (.ply / render image) after an ownership check.
GET /api/history	Returns the logged-in user's past ImageSets and ReconSessions.
DELETE /api/sessions/:id	Deletes a session's database rows and on-disk files, after an ownership check.

Technologies: Node.js, Express.js, Multer, Passport.js, jsonwebtoken, bcryptjs, Sequelize, node-fetch/axios (to call the FastAPI bridge).

Component 8 — Database & Authentication Layer (PostgreSQL) — NEW

Purpose: durably persist every user, upload, reconstruction session, and result, and enforce per-user access control. This layer replaces an earlier MongoDB-based plan; the project now uses PostgreSQL exclusively, for its relational integrity guarantees (foreign keys between User → ImageSet → ReconSession → ModelOutput → RenderedView/Metrics) which map naturally onto the pipeline's session-based structure. Full schema in Section 10.

Why PostgreSQL over the original MongoDB plan

- The data model is inherently relational — every RenderedView and Metrics row belongs to exactly one ModelOutput, which belongs to exactly one ReconSession, which belongs to exactly one ImageSet and one User. Foreign keys and cascading deletes express this far more directly than document embedding or manual reference-following.
- Session-ownership checks (a core tested requirement — Table 4.1.1, items 10–11) are simple indexed JOIN/WHERE queries in PostgreSQL rather than application-level reference validation.
- Sequelize migrations give the team a versioned, reviewable schema history, which suited the Agile/Scrum sprint structure better than a schema-less document store.

Technologies: PostgreSQL 15+, Sequelize (ORM + migrations), Passport.js (authentication strategies), JWT + bcryptjs (session/token management and password hashing).

Component 9 — Frontend (React.js + Three.js)

Purpose: provide an intuitive web interface for authentication, uploading images, viewing reconstruction progress, and interactively exploring 3D models. Built with React.js (via Vite) and Three.js; no 3D software installation is required.

Key UI Screens

Screen	Description
Landing / Login / Sign Up	Public landing page plus local and Google OAuth2 authentication.
Upload Page	Drag-and-drop or file-select interface for uploading 20–40 statue images, with preview thumbnails and project details (name, category, description).
Processing Dashboard	Live status indicator polling GET /api/sessions/:id/status, showing pipeline stages from colmap_features through nerf_training/gaussian_training to done.
Results Viewer	Side-by-side NeRF and Gaussian Splatting rendered outputs plus the PSNR/SSIM/LPIPS comparison table.
3D Model Viewer	Interactive Three.js canvas for rotating, zooming, and panning the reconstructed point cloud.
History Page	A logged-in user's past ImageSets and ReconSessions, scoped to their own account.
Account Settings	Profile fields (name, language, country), default reconstruction quality/method preference, and account deletion.

Technologies: React.js, Vite, Three.js, Tailwind CSS, Axios.

8. DETAILED PIPELINE WALKTHROUGH (pipeline.py)

8. Detailed Pipeline Walkthrough — pipeline.py

This section documents the hardened reconstruction script that FastAPI invokes as a subprocess for every job. It was rewritten during integration testing to fail loudly on missing dependencies, stream logs live instead of buffering silently, and never report a false 'done'.

8.1 Invocation

```
python pipeline.py <session_id> <statue_name> <method> [quick]
  method: nerf | gaussian | both
  quick:  optional 4th arg literal 'quick' -> tiny iteration counts for a smoke test
```

8.2 Pre-flight Checks

Before touching any images, the script verifies colmap, ns-train (for nerf/both), and CUDA availability via `torch.cuda.is_available()`. Any missing dependency aborts immediately with a clear error, instead of the job silently limping to a false 'done' after 40 minutes — the single biggest reliability fix over the original prototype script.

8.3 Live Log Streaming & Checkpoint Watching

Every external command (`colmap`, `ns-train`, `train.py`, ...) runs through a `run()` helper built on `subprocess.Popen` with line-by-line stdout streaming, rather than `subprocess.run(capture_output=True)`, which buffers all output until the process exits and looks identical to a hang during a 30–90 minute training step. A background thread polls the output directory every 5 seconds and logs CHECKPOINT SAVED: <path> (<size> MB) the instant a `.ckpt` or `point_cloud.ply` file appears, and everything is duplicated to <session>/pipeline.log for post-mortem debugging.

8.4 Stage-by-Stage Flow

Stage	% Progress	What Happens
validating	5%	Confirms the session's image folder exists and has at least 10 images.
colmap_features	10%	colmap feature_extractor with SIMPLE_PINHOLE camera model, GPU SIFT disabled for Colab stability.
colmap_matching	25%	colmap exhaustive_matcher (≤ 100 images) or sequential_matcher.
colmap_mapper	40%	colmap mapper — bundle adjustment producing sparse/0.
nerf_convert	50%	Custom parser converts sparse/0's images.txt + cameras.txt into transforms.json, strictly stepping two lines per image to avoid duplicate/garbage frames.
nerf_training	60%	ns-train nerfacto with a checkpoint watcher; iteration/save-interval scaled down for 'quick' smoke tests.
nerf_render	75%	ns-render dataset --output-format images — image frames only, no video encoding.
nerf_export	82%	ns-export pointcloud — produces <statue>_nerf.ply.

Stage	% Progress	What Happens
nerf_eval	85%	ns-eval — produces nerf_metrics.json (PSNR/SSIM/LPIPS).
gaussian_setup	78%	ensure_gaussian_splatting() — clones/builds the repo and CUDA submodules if missing.
gaussian_training	80%	train.py -s <colmap_root> --eval, with a checkpoint watcher on point_cloud.ply.
gaussian_render	90%	render.py — renders both train and test splits.
gaussian_eval	95%	metrics.py — produces gaussian_metrics.json (PSNR/SSIM/LPIPS).
done	100%	Writes results/manifest.json with per-method success flags and file paths; never reports success unless the actual output files exist on disk.

8.5 Honest Success Reporting

The original prototype wrote status: success unconditionally at the end, even when both methods had silently failed — the worst possible failure mode for a live demo, since it looks successful in the log until the actual output files are checked. The hardened script instead inspects the filesystem directly (does nerf_renders/ have files? does <statue>.ply exist? does gaussian_metrics.json exist?) and reports 'partial_failure' with a non-zero exit code if the expected artifacts are missing, so a failure is visible immediately rather than discovered later.

8.6 Environment Issues Found and Fixed During Integration

#	Module	Root Cause	Fix
1	COLMAP	Default Qt display backend requires a GUI; fails headless.	Set QT_QPA_PLATFORM=offscreen.
2	Gaussian Splatting	Rasterizer failed on non-standard COLMAP camera models.	Force SIMPLE_PINHOLE during feature extraction.
3	COLMAP → NeRF conversion	transforms.json contained ~140 duplicate/garbage frames from parsing images.txt line-by-line.	Parse in strict (i += 2) pose+feature-point pairs.
4	NeRF rendering	Render step reported 0 output frames despite training completing; glob pattern didn't match the nested output subdirectory.	Corrected the glob pattern; later moved to ns-render dataset with --output-format images.
5	Training stability	Long runs interrupted by Colab's free-tier session limits.	Checkpoint every 5,000 iterations with resume support.
6	Gaussian Splatting build	pip install -e . targeted the repo root, which has no setup.py.	Install the two submodules (diff-gaussian-rasterization, simple-knn) directly.

#	Module	Root Cause	Fix
7	NeRF training (runtime)	'Torch not compiled with CUDA enabled' — a CPU-only PyTorch build was installed on the runtime.	Reinstall a CUDA-matched torch build; pre-flight check now catches this before training starts.
8	NeRF render/export	ns-render dataset hung indefinitely after 100% frame completion, muxing an mp4.	--output-format images skips video encoding; timeout raised to 1800s as a backstop.
9	NeRF worker crash	IndexError in a background DataProcessor worker — phone-photo EXIF orientation tags caused a width/height mismatch between COLMAP's recorded intrinsics and nerfstudio's loaded pixel array.	Bake EXIF rotation into pixels (PIL ImageOps.exif_transpose) before copying images into nerf_data/images.
10	Gaussian Splatting scene path	-s pointed at sparse/0 (one level too deep) → 'Could not recognize scene type!'.	-s must be the COLMAP root, the parent of sparse/0.
11	NeRF eval OOM	Full-resolution (3024×4032) periodic eval render exhausted GPU memory mid-training.	Reduced eval image resolution / render batch size for the eval step.

9. AI Pipeline Service (FastAPI) & Node.js Bridge

The reconstruction pipeline runs on a Google Colab GPU runtime, which has no stable public address and no persistent process manager. To make it callable from the production Node.js backend, it is wrapped in a small FastAPI service and exposed publicly through an ngrok tunnel for the lifetime of the Colab session.

9.1 FastAPI Endpoints (Python, runs on Colab)

Endpoint	Description
GET /health	Liveness check used by the Node.js backend to confirm the ngrok tunnel is currently reachable before dispatching a job.
POST /jobs	Accepts session_id, statue_name, method, and the multipart image list. Validates method $\in$ {nerf, gaussian, both} and requires 5+ images (HTTP 400 otherwise). Saves images and launches pipeline.py as a non-blocking background subprocess.
GET /jobs/{session_id}/status	Returns the live contents of status.json — {stage, percent} — written by pipeline.py at every stage transition.
GET /jobs/{session_id}/results	Returns results/manifest.json once the pipeline has finished; HTTP 404 with 'not ready' while still running.
GET /jobs/{session_id}/files/{file_path}	Streams a specific result file (.ply, render .png, metrics .json) from the session's results directory.

9.2 ngrok Bridge

On session start, the Colab notebook launches the FastAPI app with uvicorn on 127.0.0.1:8000, then opens an ngrok tunnel (ngrok.connect(8000)) and prints the resulting public HTTPS URL. That URL is copied into the Node.js backend's environment as PIPELINE_URL and used as the base URL for every FastAPI call. Because the URL changes every time the Colab runtime restarts, the Node.js backend treats it as a runtime configuration value, not a build-time constant, and the /health endpoint is polled before each job dispatch to detect a stale tunnel early.

9.3 Node.js → FastAPI Call Pattern

```
const env = { ...process.env, QT_QPA_PLATFORM: 'offscreen' };
// Node.js forwards the already-validated upload to the FastAPI bridge:
await axios.post(`${PIPELINE_URL}/jobs`, formData, { headers: formData.getHeaders() });
// then polls:
await axios.get(`${PIPELINE_URL}/jobs/${sessionId}/status`);
await axios.get(`${PIPELINE_URL}/jobs/${sessionId}/results`);
```

The Node.js backend is responsible for translating each poll into a PostgreSQL write (Section 10) — the FastAPI layer itself holds no durable state beyond the session's files on the Colab Drive mount.

10. DATABASE SCHEMA (POSTGRESQL)

10. Database Schema — PostgreSQL

Shilpa3D persists all application state in PostgreSQL, accessed through Sequelize models and migrations. This section documents the entity-relationship design and the corresponding table definitions. This schema supersedes an earlier plan to use MongoDB; the project now uses PostgreSQL exclusively for all structured data.

10.1 Entity-Relationship Overview

Eight entities model the full user → upload → reconstruction → result chain:

Entity	Relationship	Cardinality
User	uploads → ImageSet	1 : M
ImageSet	triggers → ReconSession	1 : M
ReconSession	triggers → CameraPose	1 : M
ReconSession	runs → ColmapOutput	1 : M
ReconSession	produces → ModelOutput	1 : M
ModelOutput	generates → RenderedView	1 : M
ModelOutput	evaluated by → Metrics	1 : M

10.2 Table Definitions

users

Column	Type	Notes
user_id	UUID / SERIAL, PK	Primary key.
username	VARCHAR(64), UNIQUE, NOT NULL	
email	VARCHAR(255), UNIQUE, NOT NULL	Used for local login and Google OAuth2 account linking.
password_hash	VARCHAR(255), NULLABLE	bcryptjs hash; NULL for Google-only accounts.
auth_provider	VARCHAR(16), NOT NULL, DEFAULT 'local'	'local' or 'google'.
created_at	TIMESTAMP, NOT NULL, DEFAULT now()	

image_sets

Column	Type	Notes
set_id	UUID / SERIAL, PK	Primary key.

Column	Type	Notes
user_id	FK → users.user_id, NOT NULL	Cascade delete with the owning user.
statue_name	VARCHAR(128), NOT NULL	
upload_date	TIMESTAMP, NOT NULL, DEFAULT now()	
image_count	INTEGER, NOT NULL	Validated ≥ 5 at the API layer before this row is created.

recon_sessions

Column	Type	Notes
session_id	UUID, PK	Also used as the FastAPI/pipeline.py session identifier — the same ID threads through Postgres, FastAPI, and the on-disk session folder.
set_id	FK → image_sets.set_id, NOT NULL	
method	VARCHAR(16), NOT NULL	'nerf' 'gaussian' 'both'.
status	VARCHAR(32), NOT NULL, DEFAULT 'queued'	Mirrors pipeline.py's stage field; terminal values: 'done', 'error', 'partial_failure'.
start_time	TIMESTAMP, NOT NULL	
end_time	TIMESTAMP, NULLABLE	

colmap_outputs

Column	Type	Notes
colmap_id	UUID / SERIAL, PK	
session_id	FK → recon_sessions.session_id, NOT NULL	
sparse_cloud_path	TEXT, NOT NULL	Path to sparse/0.
transforms_json_path	TEXT, NOT NULL	Path to the converted transforms.json.
feature_match_count	INTEGER, NULLABLE	Diagnostic — total matched feature pairs, useful for debugging weak registration.

camera_poses

Column	Type	Notes
pose_id	UUID / SERIAL, PK	
session_id	FK → recon_sessions.session_id, NOT NULL	
rotation_matrix	JSONB, NOT NULL	3×3 rotation matrix per registered image.
translation_vector	JSONB, NOT NULL	3-element translation vector.

Column	Type	Notes
focal_length	DOUBLE PRECISION, NOT NULL	

model_outputs

Column	Type	Notes
output_id	UUID / SERIAL, PK	
session_id	FK → recon_sessions.session_id, NOT NULL	
method_type	VARCHAR(16), NOT NULL	'nerf' 'gaussian' — one row per method per session.
file_path	TEXT, NOT NULL	Path to the exported .ply.
render_preview_path	TEXT, NULLABLE	Path to a representative preview thumbnail.

rendered_views

Column	Type	Notes
render_id	UUID / SERIAL, PK	
output_id	FK → model_outputs.output_id, NOT NULL	
view_angle	DOUBLE PRECISION, NULLABLE	Camera angle for this novel-view render, where applicable.
image_path	TEXT, NOT NULL	
resolution	VARCHAR(16), NOT NULL	e.g. '1920x1080'.

metrics

Column	Type	Notes
metric_id	UUID / SERIAL, PK	
output_id	FK → model_outputs.output_id, NOT NULL	
psnr	DOUBLE PRECISION, NOT NULL	dB, higher is better.
ssim	DOUBLE PRECISION, NOT NULL	0–1, higher is better.
lpips	DOUBLE PRECISION, NOT NULL	Lower is better.
training_time_sec	INTEGER, NOT NULL	
gpu_vram_gb	DOUBLE PRECISION, NULLABLE	Peak VRAM observed during training.
render_fps	DOUBLE PRECISION, NULLABLE	Inference-time frame rate, where measured.

10.3 Sequelize Notes

- Each table above is a Sequelize model with associations mirroring the foreign keys: User.hasMany(ImageSet), ImageSet.hasMany(ReconSession), ReconSession.hasOne(ColmapOutput), ReconSession.hasMany(CameraPose), ReconSession.hasMany(ModelOutput), ModelOutput.hasMany(RenderedView), ModelOutput.hasMany(Metric).
- onDelete: 'CASCADE' is set on the ImageSet → User and ReconSession → ImageSet associations, so deleting a user or an image set cleanly removes all dependent reconstruction data (matches the tested 'session deletion' requirement in Table 4.1.1).
- Migrations are managed via sequelize-cli (sequelize db:migrate), giving the team a reviewable, versioned schema history appropriate for the project's Scrum sprint cadence.
- rotation_matrix and translation_vector are stored as JSONB rather than normalized numeric columns, since they are always read/written as a whole matrix/vector and never queried by individual component — this avoids nine extra columns per camera pose.

11. Algorithms

This section explains the key algorithms that power Shilpa3D, written accessibly so readers without a deep technical background can understand what each algorithm does and why it is used.

11.1 Structure-from-Motion (SfM)

SfM estimates the 3D structure of a scene and the camera poses simultaneously from a sequence of 2D images — the foundational step that makes everything else possible.

Step-by-Step Process

- **Feature Detection:** SIFT detects stable keypoints in each image — points that are distinctive and repeatable across viewpoints and lighting.
- **Feature Matching:** for every pair of overlapping images, the algorithm finds corresponding keypoints.
- **Fundamental Matrix Estimation:** given matches, the algorithm estimates the epipolar geometry between pairs of camera views.
- **Bundle Adjustment:** all camera poses and 3D point positions are jointly optimized to minimize reprojection error.
- **Output:** a sparse 3D point cloud and calibrated camera poses for every input image.

Used in Shilpa3D by: COLMAP, which implements the complete SfM pipeline with both sparse and dense reconstruction capabilities.

11.2 Volumetric Rendering

Volumetric rendering is how NeRF synthesizes 2D images from its learned 3D representation, simulating how light travels through a participating medium. For a given pixel, a ray is cast from the camera into the scene; N points are sampled along it; at each point the network predicts color and density; the final pixel color is the accumulation of these colors weighted by transmittance — how much light survives to that point without being absorbed. Points behind opaque surfaces contribute nothing because transmittance falls to zero after the surface is hit.

11.3 Ray Sampling Strategies

Strategy	Description
Stratified Sampling	The ray is divided into N equally-spaced bins, and one point is randomly sampled within each bin — even coverage without clustering.
Hierarchical Sampling	A coarse network first identifies regions of high density; a fine network then concentrates samples there for better detail, avoiding wasted computation in empty space.

11.4 Gaussian Rasterization

Instead of casting rays, 3D Gaussian Splatting projects Gaussian primitives directly onto the image plane — 'splatting'. Each 3D Gaussian is projected via the camera's projection matrix into a 2D ellipse; all projected Gaussians are sorted by depth; the image is divided into tiles for parallel GPU processing; Gaussians are alpha-composited in depth order, each contributing color proportional to its opacity and its footprint at that pixel. Rasterization is a well-established, highly parallel GPU operation — the same principle used in real-time game graphics — which is why it avoids the slow per-pixel ray-marching loop that makes NeRF slow.

11.5 Backpropagation & Gradient Descent

Element	Description
Loss Function	Mean Squared Error (MSE) between predicted and ground-truth pixel colors across training images.
Optimizer	Adam — an adaptive gradient-descent algorithm that adjusts the learning rate per parameter for faster, more stable convergence.
Backpropagation	PyTorch's autograd engine computes gradients of the loss w.r.t. all learnable parameters via the chain rule.
Iterations	NeRF: 30,000 iterations in the tested configuration. Gaussian Splatting: 15,000 iterations, converging in roughly a third of the wall-clock time.

12. Technologies & Tools

Comprehensive reference for every technology used in Shilpa3D. The data-layer row reflects the current PostgreSQL-based architecture; MongoDB was evaluated early in planning but is not part of the shipped system.

12.1 AI & Reconstruction Stack

Technology	Version / Type	Role in Shilpa3D
Python	3.10+	Primary language for all AI, preprocessing, and evaluation scripts.
PyTorch	2.x	Deep learning framework for the NeRF MLP and Gaussian parameter optimization; provides autograd.
CUDA	11.x+	NVIDIA GPU acceleration for tensor operations and the custom Gaussian rasterizer.
COLMAP	3.x	Structure-from-Motion software; camera pose estimation and sparse point clouds.
nerfstudio	1.x	NeRF training/eval/export/render toolkit (ns-train, ns-render, ns-export, ns-eval).
OpenCV	4.x	Image preprocessing: resizing, blur detection, normalization.
Open3D	0.17+	Point cloud processing, normal estimation for ns-export, visualization/inspection.
lpips (pip)	0.1.x	Perceptual metric backend used by Gaussian Splatting's metrics.py.
NumPy	1.24+	Array manipulation and linear algebra in preprocessing/conversion scripts.

12.2 AI-to-Backend Bridge

Technology	Role in Shilpa3D
FastAPI	Python REST API wrapping pipeline.py; runs inside the Colab GPU runtime.
uvicorn	ASGI server running the FastAPI app on 127.0.0.1:8000.
ngrok (pyngrok)	Secure public tunnel exposing the Colab-hosted FastAPI server to the Node.js backend.
Google Colab	Free-tier T4 GPU runtime used for both development and the current production reconstruction backend.

12.3 Backend & Data Stack

Technology	Version / Type	Role in Shilpa3D
Node.js	18+ LTS	Server-side JavaScript runtime; runs the Express.js API and calls the FastAPI bridge.
Express.js	4.x	REST API routes, middleware (CORS, body-parsing), error handling.
PostgreSQL	15+	Relational data store for users, image sets, sessions, outputs, renders, and metrics. Replaces the earlier MongoDB plan.
Sequelize	6.x	ORM and migration management for PostgreSQL.
Passport.js	0.7.x	Authentication strategy handling — Local strategy and Google OAuth2.
jsonwebtoken (JWT)	9.x	Access/refresh token issuance and verification for session management.
bcryptjs	2.x	Password hashing for local accounts.
Multer	1.x	multipart/form-data middleware for image uploads.
axios	1.x	HTTP client used by Node.js to call the FastAPI bridge.

12.4 Frontend Stack

Technology	Version / Type	Role in Shilpa3D
React.js	18.x	Component-based UI framework for the single-page application.
Vite	5.x	Build tool and dev server for the React app.
Three.js	r160+	WebGL-based rendering of point clouds/meshes with orbit controls.
Tailwind CSS	3.x	Utility-first CSS for layout and responsive design.
Axios	1.x	HTTP client for calling the Express.js backend.

12.5 Development & Experimentation Tools

Tool	Purpose
Visual Studio Code	Primary code editor for Python, JavaScript, and configuration files.
Jupyter Notebook / Google Colab	Interactive environment for pipeline development and GPU training experiments.
Postman	API endpoint testing for both the Node.js and FastAPI layers.
Figma	UI/UX design and prototyping.
Draw.io	System, ERD, and use-case diagrams.
Git / GitHub	Version control; team members work on separate branches (ai-pipeline, backend, frontend) merged via pull requests.

13. TESTING, RESULTS & EVALUATION

13. Testing, Results & Evaluation

Testing covered both the reconstruction pipeline (COLMAP, NeRF, Gaussian Splatting) and the supporting web application (upload, job orchestration, session history, and result retrieval).

13.1 Test Case Summary

#	Test Case	Result
1	Image upload minimum count (< 5 rejected)	OK
2	Reconstruction method validation ('nerf'/'gaussian'/'both' only)	OK
3	Image format validation (JPG/PNG only)	OK
4	Background job start returns immediately	OK
5	Progress polling reflects current stage/percent	OK
6	Results endpoint 404s before completion	OK
7	Results endpoint returns manifest with per-method success flags after completion	OK
8	File path traversal protection	OK
9	Result file (.ply / .png) download	OK
10	Session history scoped to the owning user	OK
11	Session deletion blocked for non-owners (403)	OK
12	Session deletion removes files from disk	OK
13	COLMAP produces sparse point cloud + poses	OK
14	COLMAP → transforms.json has exactly one frame per registered image	Fixed
15	NeRF training completes on GPU runtime	Fixed
16	Gaussian Splatting build & training	Fixed
17	PSNR/SSIM/LPIPS computed against held-out views	OK

13.2 Quantitative Reconstruction Quality

Metrics were computed per held-out test image and averaged across all held-out views, for each statue and each method:

Statue	Method	PSNR ↑ (dB)	SSIM ↑	LPIPS ↓	Train Time
Statue-1	NeRF	10.91	0.163	0.829	~90 min
Statue-1	Gaussian Splatting	20.80	0.577	0.511	~35 min
Statue-2	NeRF	9.64	0.123	0.857	~90 min
Statue-2	Gaussian Splatting	11.15	0.170	0.743	~35 min

Gaussian Splatting outperformed NeRF on every metric for both statues while training in roughly a third of the time. The margin was largest on Statue-1 (20.80 dB vs. 10.91 dB PSNR); on Statue-2 the gap narrowed (11.15 dB vs. 9.64 dB), which per-image inspection attributed to sparser COLMAP camera registration on Statue-2 limiting how well either method could reconstruct under-covered viewpoints — an input-quality constraint for future data collection, not a weakness specific to either method.

13.3 Qualitative Visual Analysis

Numerical scores were cross-checked against side-by-side comparisons of ground truth, NeRF output, and Gaussian Splatting output for held-out test views. NeRF's renders were visually dominated by low-frequency color noise with no recognizable statue geometry, consistent with the low PSNR/SSIM scores — 30,000 nerfacto iterations were not sufficient to converge at this dataset size and camera spread. Gaussian Splatting's renders clearly recovered the statue's silhouette, color, and surrounding scene structure, though fine detail was affected by floating-artifact streaks near the statue base in regions with fewer overlapping camera views.

13.4 System Performance

COLMAP structure-from-motion (feature extraction, exhaustive matching, incremental mapping) took approximately 20–26 minutes for a 21-image session on a T4 GPU runtime, scaling with image count and resolution. NeRF training (30,000 iterations) took approximately 90 minutes per statue; Gaussian Splatting training (15,000 iterations) took approximately 35 minutes. The full 'both' pipeline completes in roughly 2–3 hours on free-tier Colab hardware.

13.5 Summary of Findings

The pipeline meets its core objective of reconstructing 3D representations of cultural statues from unstructured photo sets. COLMAP reliably recovers camera poses and sparse geometry; Gaussian Splatting consistently outperforms NeRF on both reconstruction quality and training time at this dataset scale, making it the recommended default method going forward. All identified defects — spanning the COLMAP conversion logic, the Gaussian Splatting build process, NeRF's CUDA dependency, and the web application's session-ownership and file-access controls — were resolved and re-verified. The system is stable end-to-end, from authenticated image upload through background processing to a downloadable, database-tracked 3D output.

14. HARDWARE REQUIREMENTS

14. Hardware Requirements

Component	Specification
GPU	NVIDIA GPU with CUDA support (RTX 3060 or better for local use; free-tier Colab T4 for the current deployment). Minimum 8 GB VRAM; 12–16 GB recommended for NeRF.
RAM	Minimum 16 GB system RAM; 32 GB recommended for large image datasets.
CPU	Multi-core processor (8+ cores) for COLMAP feature extraction and data loading during training.
Storage	Minimum 50 GB free. Image datasets, COLMAP outputs, model checkpoints, and rendered outputs accumulate quickly.
OS	Linux (Ubuntu 20.04+) preferred for CUDA compatibility. Windows with WSL2 also supported.
Database	PostgreSQL 15+, can run on modest shared hosting — the data layer is not GPU- or memory-intensive.

15. Limitations

- GPU requirement: NeRF and Gaussian Splatting cannot run on CPU alone in practical time.
- Training time: NeRF may require ~90 minutes or more per statue on free-tier hardware.
- Image sensitivity: poor-quality inputs (blur, inconsistent lighting, EXIF orientation quirks) degrade reconstruction quality and can crash NeRF's dataloader if not preprocessed.
- Static objects only: the system is designed for non-moving subjects.
- COLMAP dependency: reconstruction quality is bounded by the quality of camera pose estimation, and can fail outright on small, texture-poor, or reflective sculptural surfaces.
- Free-tier Colab constraints: session disconnections, changing ngrok public URLs, and limited/queued GPU availability affect reliability and require the Node.js backend to treat PIPELINE_URL as a volatile runtime value.
- Storage growth: intermediate and final files (COLMAP outputs, checkpoints, renders) accumulate quickly; automated archiving/cleanup is not yet implemented.
- Interactive 3D viewer: still under active development, limiting direct in-browser inspection of some outputs.

16. Future Enhancements

- Move the reconstruction pipeline from free-tier Google Colab to dedicated or paid GPU infrastructure for reliability and predictable performance.
- Complete the interactive Three.js 3D viewer for full in-browser rotate/zoom/pan of reconstructed models.
- Add image-capture guidance that checks camera coverage and overlap before reconstruction begins.
- Real-time reconstruction using Instant-NGP or similar accelerated NeRF variants.
- Video-based input: extract frames from a walkthrough video instead of separate photos.
- Fallback to learned dense-correspondence pose estimation (e.g., DUST3R) for small, texture-poor, or reflective statues where standard COLMAP SfM fails.
- AR/VR export: glTF/glb output for AR applications or VR headsets.
- Automated storage management, administrator analytics, encrypted data storage, request rate limiting, and access logging.
- Expand into a centralized digital heritage archive where museums and institutions can upload, organize, and share reconstructed statues with historical metadata.

17. Conclusion

Shilpa3D demonstrates that high-quality 3D reconstruction of cultural artifacts — specifically statues — is achievable without expensive scanning hardware, using only a set of ordinary photographs and modern neural rendering techniques. By implementing and comparing two state-of-the-art reconstruction paradigms — NeRF and 3D Gaussian Splatting — the project produces both a functional reconstruction system and a rigorous, quantitatively grounded comparative analysis. On the tested dataset, Gaussian Splatting offered both faster training and higher reconstruction quality, while NeRF's underlying volumetric approach remains valuable as a comparison baseline and for future scenes with different characteristics.

The full-stack integration — a Python AI pipeline bridged through FastAPI and ngrok, a Node.js/Express.js REST API backed by a PostgreSQL/Sequelize data layer with Passport.js authentication, and a React.js + Three.js frontend — demonstrates how cutting-edge AI research can be packaged into an accessible, multi-user, persistent web application. The move from an initially planned MongoDB store to PostgreSQL reflects the genuinely relational shape of the pipeline's data: every render and metric traces back through a strict chain of ownership to a single user, and PostgreSQL's foreign keys and cascading deletes express that chain directly.

This makes Shilpa3D not only a research contribution but a practical, reproducible tool for digital heritage preservation — including, for the first time in the reviewed literature, an application of this specific NeRF-vs-3DGS comparison to Nepali cultural statues.

Project	Shilpa3D — 3D Reconstruction of Statues Using NeRF and Gaussian Splatting
Domain	Computer Vision Deep Learning Neural Rendering Full-Stack Web Development
Data Layer	PostgreSQL + Sequelize (current) — supersedes the earlier MongoDB plan
Duration	3 Months 3 Members